

6.11.6 — RSC + Streaming + Suspense + Partial Rendering

Now we're at the **last technical concept** in the 6.11.x series. 🎯

This one connects directly to the **Promises + Flight + RSC Payload** stuff we just discussed.

1. The problem Streaming solves

Imagine your Server Component needs three pieces of data:

```
export default async function Page() {
  const user = await getUser();
  const products = await getProducts();
  const recommendations = await getRecommendations();

  return (
    <>
      <Header user={user} />
      <Products products={products} />
      <Recommendations data={recommendations} />
    </>
  );
}
```

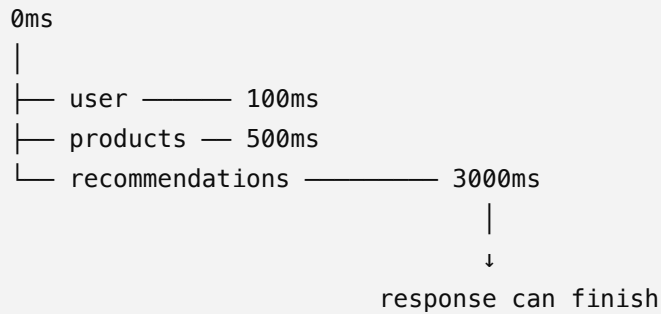
Suppose:

```
getUser()           → 100ms
getProducts()       → 500ms
getRecommendations() → 3000ms
```



Without streaming, you can end up waiting for the slow work before the relevant UI can be delivered.

Conceptually:



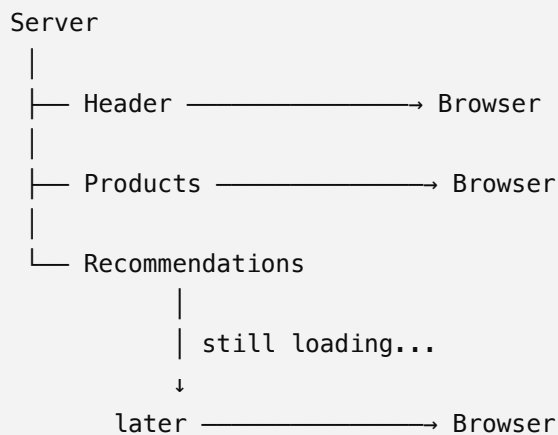
The user could be staring at a loading state while the recommendations take 3 seconds.

2. Streaming changes the idea

Instead, you can use:

```
<Suspense fallback={<RecommendationsSkeleton />}>
  <Recommendations />
</Suspense>
```

Now React can conceptually do:



So instead of:

```
WAIT FOR EVERYTHING
  ↓
SEND EVERYTHING
```

you can have:

SEND WHAT'S READY

↓

SEND MORE WHEN READY

↓

SEND MORE WHEN READY



That's **streaming**.

3. Where does Flight fit?

This is where our previous lessons connect beautifully.

Remember:

Server Component tree

↓

React Flight

↓

RSC Payload



Flight can stream the RSC information progressively.

Conceptually:

Flight stream

chunk 1 → Browser

chunk 2 → Browser

chunk 3 → Browser

chunk 4 → Browser



So:

Streaming isn't a separate alternative to Flight. Flight provides the mechanism by which RSC results can be streamed to the client.

4. Where does Suspense fit?

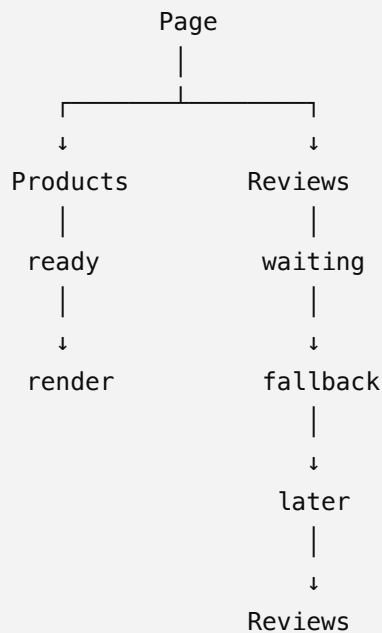
Suspense tells React:

"This part of the tree may not be ready yet. Here's what should be shown while we're waiting."

For example:

```
<Suspense fallback={<Loading />}>
  <ProductReviews />
</Suspense>
```

Conceptually:



5. So Suspense and Streaming are different

This is an important interview distinction.

Suspense

Defines a **boundary**:

"This part can wait."

Streaming

Defines how the server can **deliver results progressively**:

"Send the ready parts now,
send the waiting parts later."

Together:

Suspense

↓

defines boundaries
↓
Streaming
↓
progressive delivery

6. Example

Imagine:

```
export default function Page() {  
  return (  
    <main>  
      <Header />  
  
      <Products />  
  
      <Suspense fallback={<ReviewsSkeleton />}>  
        <Reviews />  
      </Suspense>  
    </main>  
  );  
}
```

Suppose:

Header → ready
Products → ready
Reviews → slow



The user might initially see:

Header

Products

Reviews skeleton



Then later:

Header



Products
Actual Reviews

The page didn't need to wait for Reviews before displaying the rest.

7. Is this "partial rendering"?

Here's where terminology can get confusing.

Don't equate:

Streaming

=

Partial Rendering



They aren't identical.

Streaming means results can be delivered progressively.

Partial rendering generally refers to updating/rendering only relevant portions rather than regenerating/replacing an entire application/page in a particular context.

With App Router navigation, RSC and nested layouts allow the framework to update relevant route segments while preserving other parts of the UI.

So:

RSC

+

nested layouts

+

client navigation

+

React reconciliation



helps enable granular UI updates.

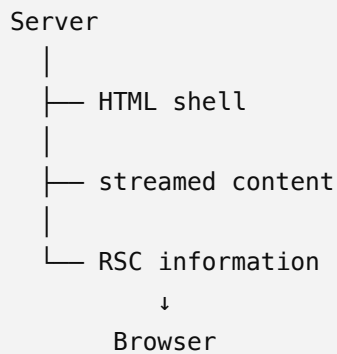
8. Initial page load vs navigation

This distinction is 🔥 important.

Initial load

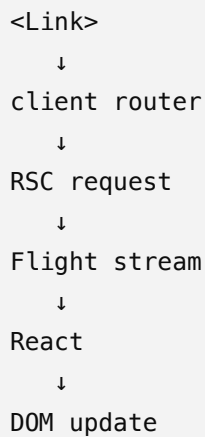
The server can stream the initial response.

Conceptually:



Client navigation

You're already inside the application.



The RSC response itself can also be streamed.

9. What happens with a slow Server Component?

Suppose:

```
async function Recommendations() {  
  const data = await getRecommendations();  
  
  return <div>{data}</div>;  
}
```

and:

```
<Suspense fallback={<Loading />}>  
  <Recommendations />  
</Suspense>
```

Conceptually:

Request
↓
Server starts rendering
↓
Recommendations suspends
↓
Suspense boundary says:
"show fallback"
↓
Flight/HTML can continue
↓
Browser receives available content
↓
Recommendations resolves
↓
server sends additional result
↓
React replaces fallback

That's the basic mechanism.

10. And now connect this to RSC Payload

The payload isn't necessarily:

ONE HUGE RESPONSE

It can be thought of as a stream:

RSC Payload Stream

Header information
Product tree
Suspense info
Reviews result



↑
arrives progressively

That's why earlier I described Flight as being designed around **streaming**.

11. What happens in the browser?

The browser doesn't necessarily wait for the entire Flight stream before React can process it.

Conceptually:

```
Flight chunk 1
  ↓
React processes it

Flight chunk 2
  ↓
React processes it

Flight chunk 3
  ↓
React processes it
```



Eventually:

```
Suspense fallback
  ↓
actual content
```



So React can progressively construct/update the UI.

12. This is also why the RSC Payload isn't just "page data"

If the payload were simply:

```
{
  "products": [...]
}
```

it wouldn't contain enough information to describe the full React Server Component result and its relationships.

Flight can communicate:

```
tree structure
+
serialized values
+
client references
+
async/suspense relationships
+
other React-specific information
```



That's why we're studying Flight before moving on to the next major topic.

13. Connect this to your Web Performance Analyzer project

This is actually useful for your planned project. 😊

Imagine:

Web Performance Analyzer



```
URL
↓
Server Component
↓
PageSpeed API
↓
analysis
↓
RSC result
↓
UI
```

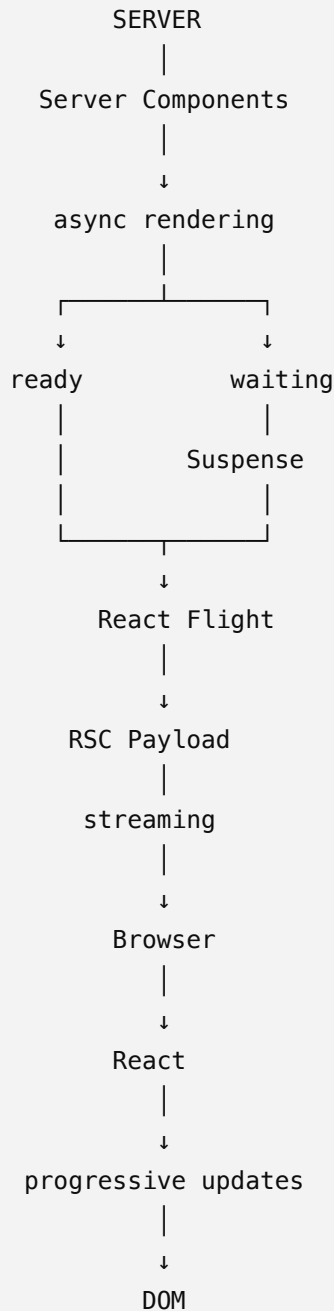
If some expensive analysis takes time, you could conceptually structure:

```
<Suspense fallback={<AnalysisSkeleton />}>
  <AnalysisResult />
</Suspense>
```

So the page could progressively reveal sections rather than making the user wait for everything.

14. The complete chain

Now look at the entire thing:



🔥 That's the relationship:

Suspense → defines what can wait

Streaming → delivers available results progressively

Flight → transports the RSC representation

RSC Payload → carries that representation

React → processes it and updates the UI

Interview answer

If they ask:

"How do Suspense and streaming work with React Server Components?"

You can say:

"Suspense lets us define boundaries around parts of the component tree that may not be ready yet. React can then stream the available result progressively rather than waiting for every async Server Component to finish. With Server Components, the Flight protocol carries the RSC representation, so the browser can receive and process the server-rendered result progressively and React can replace Suspense fallbacks as the corresponding content becomes available."

That's enough for the interview. You don't need to explain the internal Flight record format unless they specifically ask.

6.11.x — ONE MORE TO GO

We've now covered the **technical pieces** of 6.11.

Only:

6.11.7 — Complete RSC/Flight recap + interview questions

Then:

6.11.x
↓
 COMPLETE
↓
NEW CHAT
↓
6.12 — Serialization,
Client References &
Server References



And yes — after **6.11.7**, we stop this chat and start **6.12 in a fresh chat**, exactly as you requested. 🚀



ok

😊 Perfect. Next we'll do **6.11.7 — the complete RSC + React Flight recap and interview questions.**

After that, **6.11.x is officially done 🏁**, and we'll start **6.12 in a new chat.**



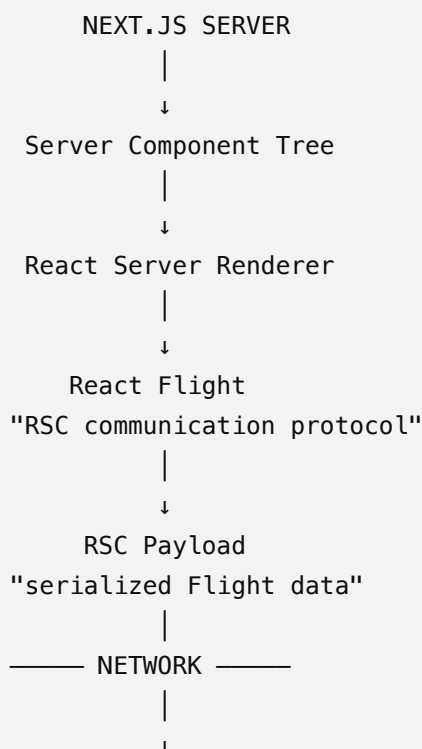
sure go ahead

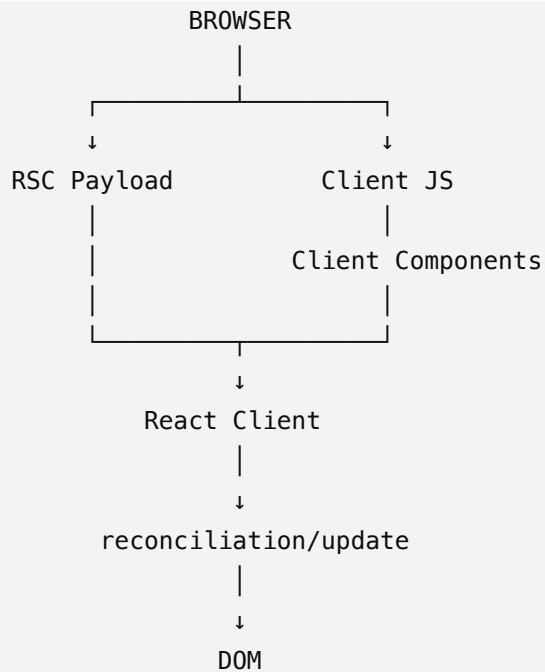
🏁 6.11.7 — Complete RSC Payload + React Flight Recap

Alright 😊 this is the **final 6.11.x lesson**. After this, we're done with RSC/Flight fundamentals and **6.12 will start in a fresh chat**, as planned.

Let's consolidate everything into one mental model first.

1. The entire RSC story in one picture





The biggest thing to remember:

Server Components don't need to be shipped to the browser as JavaScript. Their rendered result is represented in the RSC payload.

Client Components are different because their JavaScript needs to be available in the browser for interactivity.

2. Flight vs RSC Payload

This was one of your original questions.

React Flight

Think:

The protocol/mechanism used by React to transport Server Component information.

RSC Payload

Think:

The actual serialized data produced using that mechanism for a particular render/navigation.

So:

Flight

↓



protocol / mechanism

RSC Payload

↓

actual Flight data

Don't say:

"Flight is the payload."



Say:

"The RSC payload is transmitted using React Flight."



3. What does "serialized representation" mean?

Suppose the server has:

`</> TypeScript`



```
<Product
  name="Dosa"
  price={80}
/>
```

React has an internal representation of this component tree.

That internal representation can't simply be sent directly over HTTP.

So React converts the relevant information into a **transferable Flight representation**.

React tree

↓

serialization

↓

Flight representation

↓

network

↓

client React



So:

Serialized representation = a network-transferable representation of the Server Component tree, supported values, and relevant references.

And importantly:

Flight serialization

≠

JSON.stringify()



Flight has React-specific semantics.

4. Client References

Suppose:

```
</> TypeScript

'use client';

export default function AddButton() {
  return <button>Add</button>;
}
```



The Server Component tree can contain:

```
</> TypeScript

<AddButton />
```



The server doesn't need to send the component's source code inside the RSC payload.

Instead, Flight can represent a **Client Reference**.

Conceptually:

```
Server
  |
  | "Client Component X goes here"
  ↓
Client Reference
  |
  ↓
Browser
  |
  ↓
client JS module
```



↓
AddButton

So:

Client Reference

≠

Client Component source code



It's a reference to the client-side module/export.

5. Client JS vs RSC Payload

Don't mix these up.

BROWSER



RSC data
tree/results
references

Client JS
component code
event handlers

The two work together.

RSC Payload

Answers roughly:

"What Server Component result/tree should React use?"

Client JS

Provides:

"Here is the executable code for Client Components."

6. Server Functions / Server References

Now reverse the direction.

```
</> TypeScript

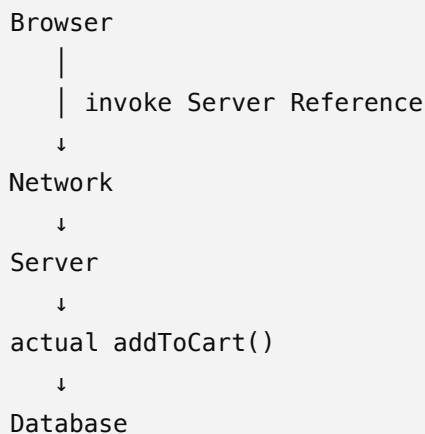
'use server';

export async function addToCart(productId) {
  await db.cart.add(productId);
}
```

The browser doesn't receive the implementation.

Instead, it can receive/use a **Server Reference**.

Conceptually:



And remember the important distinction:

```
function that happens to execute on server
    ≠
Server Function
```

A Server Function has explicit Server Function semantics, typically via:

```
</> TypeScript

'use server';
```

7. Ordinary function vs Server Function

Ordinary server-side function

</> TypeScript



```
function getProducts() {  
  return db.products.findMany();  
}
```

It may execute on the server.

But it isn't automatically remotely invokable.

server-side function



↓
runs on server
↓
ordinary function

Server Function

</> TypeScript



```
'use server';  
  
export async function addToCart() {  
  ...  
}
```

Server Function



↓
Server Reference
↓
can be invoked through supported server-action mechanisms

8. Serialization boundary

Suppose:

</> TypeScript



```
export default function Page() {  
  const product = {  
    id: 1,  
    name: "Dosa"  
  };  
};
```

```
return <ProductCard product={product} />;
}
```

and:

```
</> TypeScript
'use client';

function ProductCard({ product }) {
  ...
}
```

The `product` data crosses:

```
Server
|
| serializable value
↓
Client boundary
|
↓
Client Component
```

But an ordinary server function cannot simply be passed the same way:

```
</> TypeScript

<ProductCard
  onSave={someServerFunction}
/>
```

unless that function has the appropriate **Server Function/reference semantics**.

9. 'use client' vs 'use server'

Keep this distinction very clear.

'use client'

```
'use client'
↓
Client Component boundary
↓
client module graph
```

↓
browser JS

'use server'

'use server'
↓
Server Function semantics
↓
Server Reference
↓
remote invocation



And:

'use client' does **not** simply mean "this component only renders in the browser."

That's a common oversimplification.

10. RSC navigation

Now suppose:

/products



and:

</> TypeScript
`<Link href="/products/123">`



is clicked.

The important flow is:

<Link>
↓
Next.js Client Router
↓
RSC request
↓
Server Components execute/render
↓
React Flight
↓
RSC Payload



↓
Browser
↓
React reconciliation
↓
DOM update

Notice:

No full HTML document replacement is required for this client-side navigation.

11. DOM can change without replacing the document

This was another question you asked earlier.

Suppose:

Before

```
<main>  
  <h1>Products</h1>  
  <ProductList />  
</main>
```



After navigation

```
<main>  
  <h1>Product 123</h1>  
  <ProductDetails />  
</main>
```



The DOM can change substantially.

But:

```
DOM changed  
  ≠  
whole HTML document replaced
```



React can update the existing DOM based on the new RSC result.

That's a **very important App Router concept**.

12. Router Cache

The RSC payload received during navigation can be used by the App Router and stored in its client-side Router Cache.

Conceptually:

```
Server
  ↓
RSC Payload
  ↓
Router Cache
  ↓
React
  ↓
DOM
```



Router Cache is:

Client-side cached RSC route/segment information used by the App Router.

Don't confuse it with:

```
Data Cache
Full Route Cache
browser HTTP cache
JS bundle cache
```



They're different things.

13. Cache HIT vs MISS

HIT

```
Navigation
  ↓
Router Cache
  ↓
HIT ✅
  ↓
reuse cached RSC information
  ↓
React
  ↓
DOM
```



MISS

Navigation



Router Cache



MISS ❌



RSC request



Server



Flight



RSC Payload



Router Cache



React



This is the core navigation-caching model.

14. Prefetching

A `<Link>` can also be prefetched in applicable situations.

Conceptually:

`<Link href="/products/123">`



prefetch



RSC Payload



Router Cache



Then:

User clicks



cached RSC information



React



UI



So a request can happen **before the user actually navigates**.

15. Streaming + Suspense

Now connect the last part.

Suppose:

```
Header      → fast
Products    → fast
Reviews     → slow
```



With:

```
</> TypeScript

<Suspense fallback={<ReviewsSkeleton />}>
  <Reviews />
</Suspense>
```



React can conceptually produce:

```
Header
  ↓
Products
  ↓
ReviewsSkeleton
```



then later:

```
Reviews
  ↓
replace fallback
```



Flight allows the RSC result to be **streamed progressively**.

So:

```
Suspense
  ↓
defines boundary

Streaming
  ↓
delivers results progressively
```



Flight

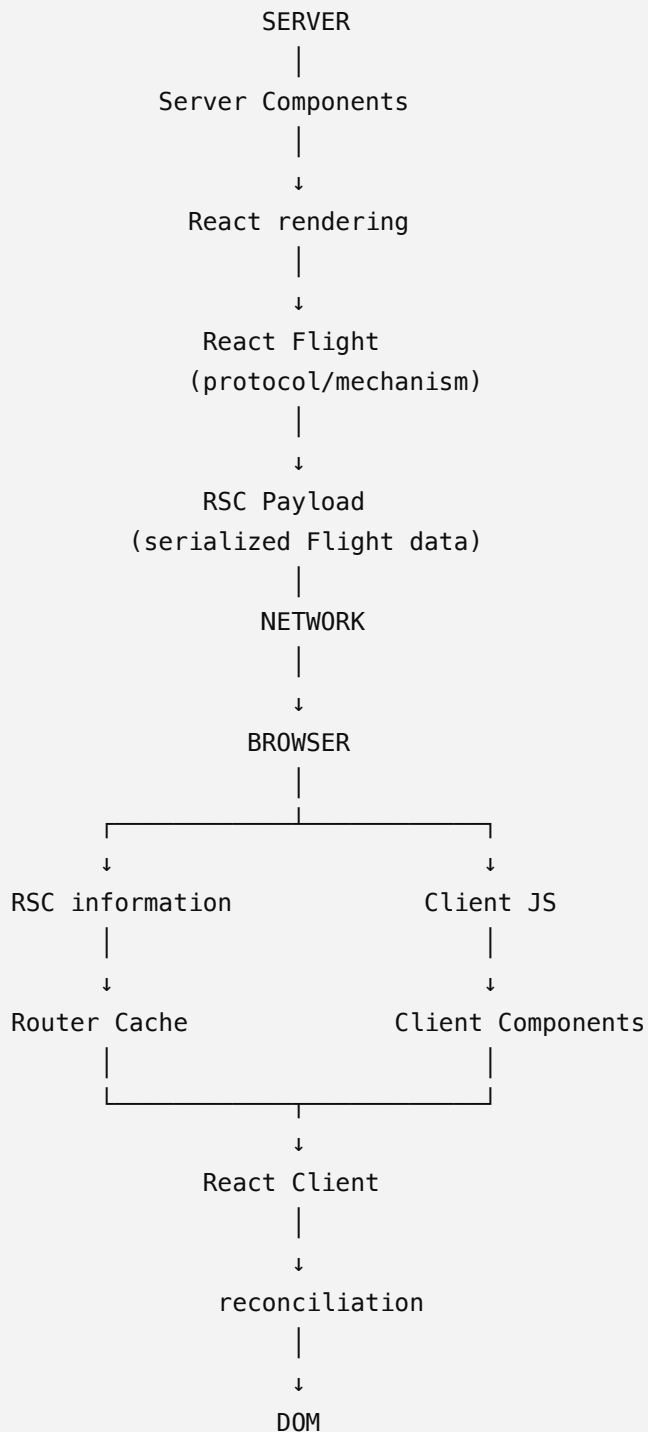


transports RSC representation

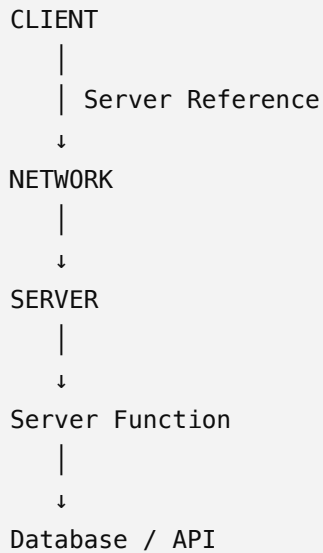
These aren't interchangeable concepts.

16. The complete architecture 🔥

Here's your final **6.11 mental model**:



For server mutations:



🎯 17. Interview Questions

Now let's test whether you've actually got it rather than just nodding along. 😂

Q1. What is React Flight?

Answer:

React Flight is the protocol/mechanism React uses to communicate the Server Component tree between server and client.

Q2. What is the RSC Payload?

The RSC Payload is the serialized Flight data representing the Server Component result/tree, supported values, and relevant references for a particular render or navigation.

Q3. Is RSC Payload HTML?

No.

HTML
→ document/UI representation

RSC Payload
→ React Server Component representation

Q4. Is RSC Payload JSON?

Not simply.

Flight has its own serialization format and semantics beyond ordinary JSON serialization.

Q5. Does the browser receive Server Component source code?

No.

The Server Component executes on the server, and its result is represented in the RSC payload.

Q6. Does the browser receive Client Component JavaScript?

Yes.

Client Components need client-side JavaScript for their interactivity.

Q7. What is a Client Reference?

A reference in the RSC system to a Client Component/module that the browser can resolve to its client-side code.

Q8. What is a Server Reference?

A reference to a Server Function that allows the client to request its invocation without receiving the function implementation.

Q9. Can any function become a Server Function just because it runs on the server?

No.

runs on server

≠



Server Function

Server Function semantics are explicitly established, typically with `'use server'` .

Q10. During `<Link>` navigation, does Next.js normally download a complete new HTML document?

No.

For client-side App Router navigation, Next.js can request the RSC payload and use it to update the existing React tree and DOM.

Q11. Can the DOM structure change during client navigation?

Absolutely.

```
existing DOM
  ↓
React receives new RSC result
  ↓
reconciliation
  ↓
new DOM structure
```



But the entire browser document doesn't have to be replaced.

Q12. What does Router Cache store?

Client-side RSC route/segment information used by the App Router for navigation and reuse.



18. Your 30-second interview answer

If they ask:

"Explain React Server Components and RSC Payload in Next.js."

You can say:

"Next.js uses React Server Components to execute server-side component logic. React's Flight protocol provides the mechanism for representing and

transporting the Server Component result to the client as an RSC payload. The payload contains serialized component-tree information, supported values, and references to Client Components. The client combines that information with the Client Component JavaScript and React uses it to construct or reconcile the UI. During App Router navigation, the RSC payload can be fetched and stored in the client Router Cache, allowing the existing application tree to be updated without a full document reload."

That's a very solid mid-level answer. 🎯

🚩 6.11.x — COMPLETE

You now have:

6.11 RSC Payload + React Flight

- |
- ├ Flight
- ├ RSC Payload
- ├ serialization
- ├ Client References
- ├ Server References
- ├ Server Functions
- ├ RSC navigation
- ├ Router Cache relationship
- ├ prefetching
- ├ streaming
- └ Suspense

🚩 DONE

And now we STOP this chat. 🙅

As planned, don't start 6.12 here.

Open a new chat and say:

"Continue Next.js 6.12"

Then we'll begin the next major piece:

🚀 6.12 — Serialization, Client References & Server References

We'll go deeper there, without mixing it back into 6.11. 😊



server function vs server side function invocation and diff

Yes — this distinction is **super important**, because "server-side function" and "Server Function" sound almost identical. 😊

1. Server-side function

A **server-side function** simply means:

A normal function whose code executes on the server.

Example:

```
</> TypeScript

async function getProducts() {
  const products = await db.products.findMany();
  return products;
}

export default async function Page() {
  const products = await getProducts();

  return <ProductList products={products} />;
}
```

Here:

```
Page()
  ↓
getProducts()
  ↓
Database
```

`getProducts()` runs on the server.

But it's just a **normal function call**:

```
Page()
  |
  | normal JS call
  ↓
getProducts()
```

There is **no network request between Page and getProducts**.

The function isn't remotely callable by the browser.

2. Server Function

A **Server Function** is different.

It is explicitly given Server Function semantics, typically with:

```
</> TypeScript

'use server';

export async function addToCart(productId) {
  await db.cart.add(productId);
}
```



Now the function can be exposed through a **Server Reference**.

Conceptually:

```
Browser
  |
  | invoke Server Reference
  ↓
NETWORK REQUEST
  ↓
Server
  ↓
addToCart(123)
```



So there's a fundamental difference:

	Server-side function	Server Function
Runs on server	✓	✓
Normal JS function	✓	Also a function, but special semantics
'use server'	✗	✓
Callable remotely from client	✗	✓ through supported mechanisms

	Server-side function	Server Function
Server Reference	✗	✓
Network involved when called from server code	✗	Not necessarily
Designed for client → server invocation	✗	✓

3. "Server-side function invocation" means something different

Suppose:

`</> TypeScript`

```
async function getUser() {  
  return db.user.find();  
}  
  
async function Page() {  
  const user = await getUser();  
}
```



The invocation is:

```
Page()  
|  
| function call  
↓  
getUser()
```



Everything is happening inside the **same server execution environment**.

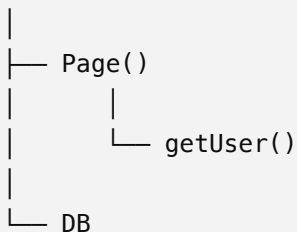
There's no:

```
browser  
↓  
network  
↓  
server
```



It's simply:

Server



4. Server Function invocation is different

Now:

`</> TypeScript`



```
'use server';

export async function addToCart(productId) {
  await db.cart.add(productId);
}
```

And the browser wants to invoke it.

The browser doesn't have:

`</> JavaScript`



```
addToCart() // implementation
```

Instead it has a reference.

Conceptually:

Client

```
Server Reference: XYZ
arguments: 123
```

↓

Network

↓

Server

```
resolve XYZ
```

↓

`addToCart(123)`



That's a **remote invocation**.

5. Here's the easiest way to remember it 🧠

Ask:

"Who is calling the function?"

If it's:

```
Server code
  ↓
Server function
```



that's just a normal function invocation, even if the function is a Server Function.

No special network trip is needed.

But if:

```
Browser
  ↓
Server Function
```



then the browser can't directly execute it.

It invokes the **Server Reference**, which causes communication with the server.

6. This is the subtle point

A Server Function doesn't mean:

"Every time this function is called, a network request happens."



It means:

The function has semantics that allow it to be referenced and invoked from the client.

So:

```
</> TypeScript

'use server';

export async function addToCart(id) {
  ...
}
```



If server code does:

```
</> TypeScript

await addToCart(123);
```



that's simply:

```
Server
  ↓
normal function invocation
  ↓
addToCart()
```



But when the **client** invokes it through its **Server Reference**, then:

```
Client
  ↓
Server Reference
  ↓
network
  ↓
Server Function
```



7. One more example

Normal server-side helper

```
</> TypeScript

async function calculatePrice(productId) {
  const product = await db.products.find(productId);
  return product.price * 1.18;
}
```



Used by:

</> TypeScript



```
export default async function Page() {  
  const price = await calculatePrice(123);  
  
  return <div>{price}</div>;  
}
```

Flow:

Page()

↓

calculatePrice()

↓

DB



No client involvement.

Server Function

</> TypeScript



```
'use server';
```

```
export async function purchase(productId) {  
  await db.orders.create({ productId });  
}
```

Client:

</> TypeScript



```
'use client';
```

```
import { purchase } from './actions';
```

```
export function BuyButton({ productId }) {  
  return (  
    <button onClick={() => purchase(productId)}>  
      Buy  
    </button>  
  );  
}
```

Conceptually:

CLICK
↓
purchase(123)
↓
Server Reference
↓
NETWORK
↓
Server
↓
actual purchase(123)
↓
DB



The interview distinction

If they ask:

"What's the difference between a server-side function and a Server Function?"

Say:

"A server-side function is simply a normal function that executes in the server environment. It can be called by other server code through an ordinary JavaScript function call. A Server Function has explicit Server Function semantics, typically through `'use server'`, which allows it to be represented by a Server Reference and invoked from client code through the server-action mechanism. The function implementation remains on the server."

And the one-liner:

Server-side function
= where it executes

Server Function
= special semantics for server invocation from the client



That distinction is exactly what I want you to carry into 6.12.



ChatGPT can make mistakes. Check important info.